



Jose Parreño Garcia <joparga3@gmail.com>

[TEST k74 - Paid] Google released TabFM: A real step forward for Tabular foundational models (but not a revolution)

1 mensaje

Jose Parreño Garcia from Senior Data Science Lead <joseparreogarcia@substack.com>
Para: joparga3@gmail.com

5 de julio de 2026 a las 21:37

[View in browser](#)

The Senior Data Science Lead newsletter

Hi! It's Jose from Senior Data Science Lead newsletter, where I write about senior data science leadership, sharing hard-earned lessons from building teams, systems, and real-world machine learning.

Because you are a paid subscriber, you are directly supporting this work — and you get full access to the practical tools built alongside the writing:

- ***The full Data Storytelling course, focused on turning analysis into insights that land with executives. Because today, anyone can build a chart with AI, but very few people can build the right chart.***



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

- ***50+ Notion templates for hiring, onboarding, mentoring, and planning built for Data and Tech Leads. As a paid subscriber, you get 50% off with code PAIDSUBS-03PL.***



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)

And as always, thank you for supporting the newsletter — your support is what keeps it independent, practical, and focused on real-world impact.

Google released TabFM: A real step forward for Tabular foundational models (but not a revolution)

Why tabular foundation models are finally worth trying, where they challenge XGBoost, and why boosted trees are not dead yet.

JOSE PARREÑO GARCIA

JUL 5 · PAID · DRAFT





READ IN APP ↗

On June 30, 2026, [Google Research published TabFM](#), a tabular foundation model released with a scikit-learn-compatible API, pretrained weights, and a GitHub repo.

To understand why this release is worth taking seriously, you have to understand what it is pushing against. Tabular data is the format that most practising data scientists actually work with every day. For roughly a decade, the algorithm that won on that format, consistently, was gradient-boosted decision trees. XGBoost, LightGBM, CatBoost. Deep learning also got its space, specially when dealing with lots of data and high cardinality interactions.

With the advent of the Transformers architecture, there a current of thought that tabular models could be built on top of a generalist foundational layer to re-use for any problem. But, the results were not encouraging.

Now, Google's TabFM is not another "let us apply a transformer to CSV files" attempt. The framing has shifted and the approach is to pretrain a model across many tasks so it learns a reusable prediction procedure, *but then*, at inference time, **treat the training rows as context** rather than gradient signal. We will cover details later in the post, but this training paradigm is different to other tabular foundational models and it's worth diving a bit deeper into it.

After having read about it, it personally feels that TabFM packages the field's best ideas into something practitioners can finally try (although with some calibrated optimism, not hype yet).

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed

Where can you find the blog post and the relevant git repo?

The blog post “[Introducing TabFM: A zero-shot foundation model for tabular data](#)”, by Weihao Kong and Abhimanyu Das, Research Scientists, Google Research was released in Google research blog space.

It comes with a companion [Git](#) and [Hugging face](#) link.

What will we cover in this post?

- **Why did gradient-boosted trees dominate tabular ML for so long?**

The peer-reviewed evidence for GBDT dominance and the 3 specific reasons neural networks struggled.

- **Why did early deep learning models for tables mostly fail?** TabNet, TabTransformer, SAINT, FT-Transformer — what went wrong and why.
- **What is a tabular foundation model, and why is the framing different?** The conceptual shift from per-dataset training to in-context learning.
- **How did we get from TabPFN to Google TabFM?** The 4-branch family tree and what each branch contributed.
- **How does Google TabFM actually work?** The exact 4-stage architecture, what `.fit()` does and does not do, and the training setup.
- **What is TabFM-Ensemble, and why does it matter for reading the benchmark?** Why the headline numbers and the zero-shot numbers are not the same thing.
- **What does the TabArena benchmark evidence actually show for TabFM?** TabArena, independent evaluations, and where the gaps are.
- **When should you reach for TabFM, and when should you stick with XGBoost?** A practical framework for the decision.
- **Where does tabular ML and foundation model research go from here?** What the trajectory of this field suggests about the next few years.

Let's get started!

Why did gradient-boosted trees dominate tabular ML for so long?

Instead of trusting my thoughts and notes about boosted trees, I wanted to rely on [Grinsztajn et al. \(NeurIPS 2022\) paper “Why do tree-based models still outperform deep learning on tabular data?”](#) to help us provide some solid data on boosted tree performance. Grinsztajn ran a systematic benchmark across 45 datasets, applied extensive hyperparameter tuning to both tree-based models and neural networks, and found that tree-based methods remained state-of-the-art on ***medium-sized tabular*** data. Even setting aside the speed advantage trees have, trees were more accurate.

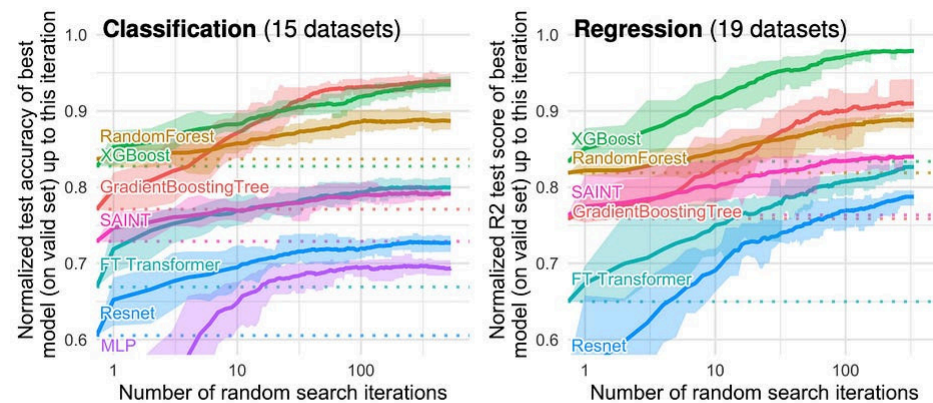


Figure 1: **Benchmark on medium-sized datasets, with only numerical features.** Dotted lines correspond to the score of the default hyperparameters, which is also the first random search iteration. Each value corresponds to the test score of the best model (on the validation set) after a specific number of random search iterations, averaged on 15 shuffles of the random search order. The ribbon corresponds to the minimum and maximum scores on these 15 shuffles.

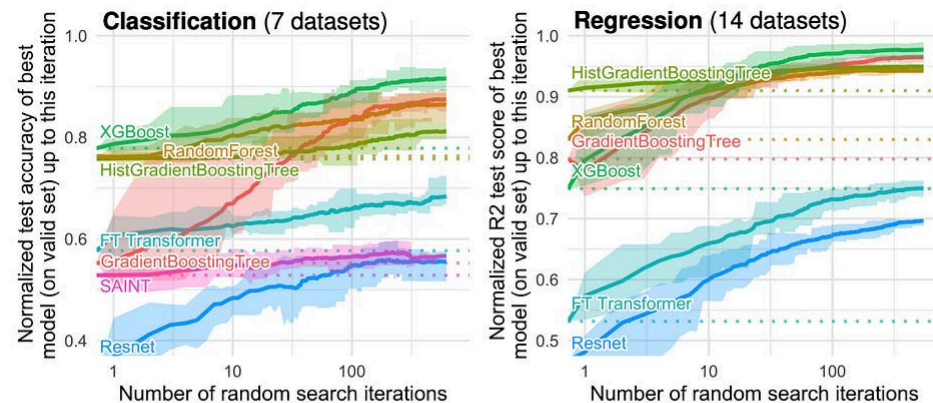


Figure 2: **Benchmark on medium-sized datasets, with both numerical and categorical features.** Dotted lines correspond to the score of the default hyperparameters, which is also the first random search iteration. Each value corresponds to the test score of the best model (on the validation set) after a specific number of random search iterations, averaged on 15 shuffles of the random search order. The ribbon corresponds to the minimum and maximum scores on these 15 shuffles.

Screenshot taken from Grinsztajn paper. Its pretty cool how 1 single image can encapsulate all the learnings of boosted trees performance against deep learning frameworks.

The Grinsztajn's goes further than just reporting the results of performance differences. It also identifies 3 structural reasons neural networks struggled on this format:

1. **Robustness to uninformative features.** Real datasets contain columns that provide no signal for the target — noise variables, proxies, redundant encodings. Decision trees handle these gracefully because splits on uninformative features are not chosen. Neural networks, particularly early tabular architectures, tend to overfit to noise, especially with limited data. Adding regularisation helps, but the benefit is inconsistent.
2. **Preserving feature orientation.** Trees make axis-aligned splits: *“is column A greater than threshold T ?”* This is a natural fit for tabular data, where many relationships between features and targets are genuinely piece-wise or threshold-like. Neural networks learn smooth, rotation-invariant functions. That is a strength in vision and language, where rotating a representation does not destroy its meaning. In tabular data, where each column has a fixed, domain-specific interpretation, rotation-invariance is a liability.
3. **Learning irregular functions.** Tabular targets often depend on the data through highly irregular, non-smooth mappings. A tree ensemble can represent those patterns directly through its split structure. Fitting them with gradient descent on a neural network requires the architecture to carve out the right function class, which takes more data, more regularisation, and more tuning to get right.

Now, bare in mind that this was a 2022 paper, and lots has happened in the past few years related with neural networks and the transformer architecture. But the underlying foundations of these 3 reasons still hold. Dont get me wrong, deep learning **can and does** beat gradient boost trees for tabular data, but only if conditions are relevant (massive datasets, high cardinality of categorical features requiring embeddings, etc).

Why did early deep learning models for tables mostly fail?

The architectures published between roughly 2019 and 2022 represent serious research.

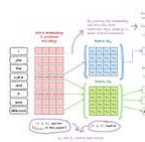
- [TabNet \(2019\)](#) introduced sparse sequential attention to select features at each decision step.
- [TabTransformer \(2020\)](#) applied self-attention to categorical embeddings.
- [SAINT \(2021\)](#) used inter-sample attention in addition to feature-level attention — it literally attends across rows as well as columns.
- [FT-Transformer, introduced by Gorishniy et al. \(NeurIPS 2021\)](#), built a clean, well-tuned Transformer adaptation for tabular data and also introduced a strong ResNet baseline.

The FT-Transformer paper clearly described how the new architecture was competitive with the best tree-based methods on some datasets, but no single deep learning architecture was universally superior.

Why did these models struggle despite genuine architectural sophistication?

The fundamental problem is that tabular columns are heterogeneous in ways that break the assumptions that make transformers work so well in NLP.

In language, a token embedding carries a relatively stable meaning across documents. The word “bank” can mean different things in different sentences, but the token itself is drawn from a consistent vocabulary and its representation can be trained to be general. If you want a deeper dive into the Transformers architecture focused on language (the tech behind LLMs), check my series about it.



Understanding Transformers (Part 4): Attention is just a few matrices

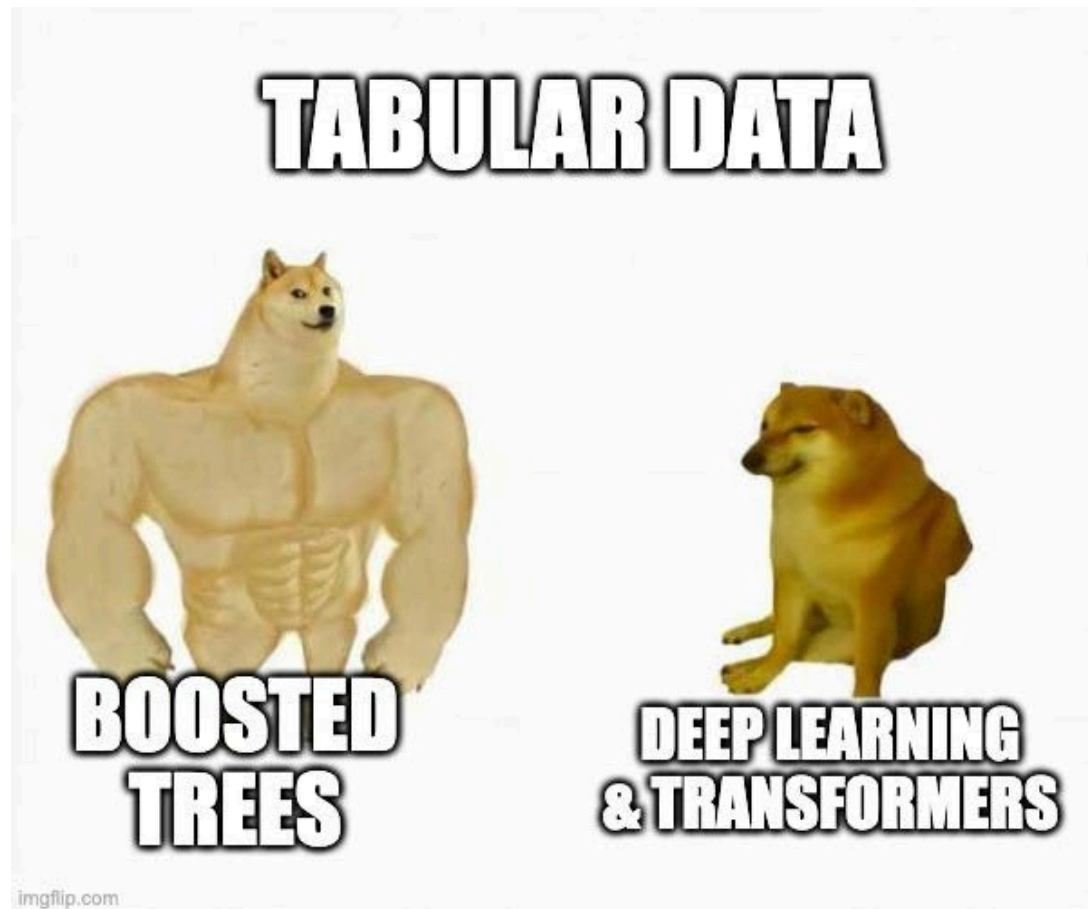
JOSE PARREÑO GARCIA · JUN 20

[Read full story](#)

In tabular data, a column called “value” in one dataset might represent dollars, in another a sensor reading, in a third a count of events. The number 1000 in a revenue column has a completely different statistical relationship to the target than the number 1000 in a latitude column. The same value in the same column across two different tables may mean entirely different things.

And herein live the problem: a model trained on one tabular dataset cannot straightforwardly transfer features learned there to another dataset. Each fit starts from scratch. That eliminates the key advantage pretraining provides in NLP: the ability to build shared representations that generalise across tasks.

TabNet, TabTransformer, SAINT, and FT-Transformer are genuinely useful papers. They clarified what was possible with deep learning on tables and set up the benchmarks that later work would beat. But they were still fundamentally per-dataset models — fit, tune, predict — and on most well-tested real datasets, a well-tuned XGBoost was at least competitive and usually faster.



What is a tabular foundation model, and why is the framing different?

Classical deep learning is based on gradient descent on your data

A classical neural network starts almost from scratch on your dataset. You give it your rows, your columns, and your target. Then, through gradient descent, it slowly adjusts its weights until it learns the patterns in that specific table.

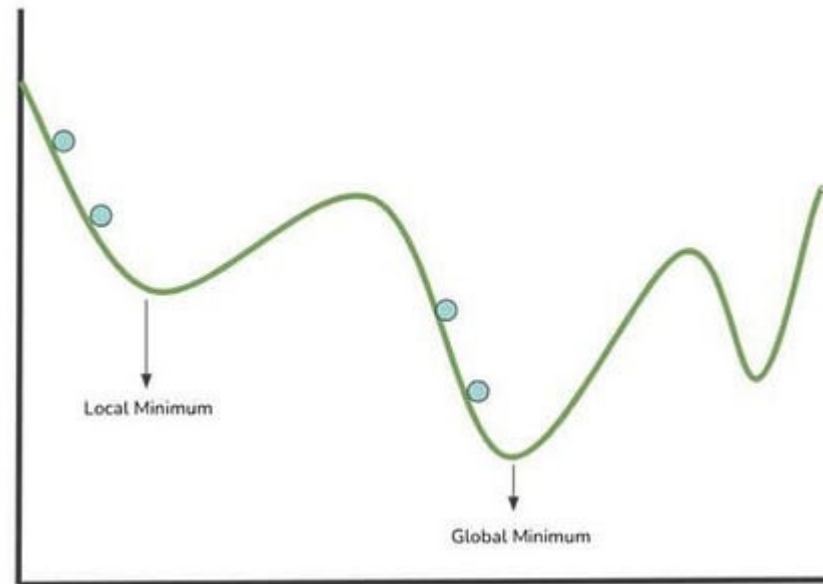


Image from [NVIDIA blog post](#).

For example, suppose you give the model a small table like this:

Bedrooms	Size	Distance to centre	Neighbourhood	Sold price
2	65 m ²	1 km	Central	£420k
3	90 m ²	3 km	Residential	£510k
1	45 m ²	1 km	Central	£310k
4	130 m ²	8 km	Suburb	£620k
2	70 m ²	7 km	Suburb	£330k

Then you ask it to predict this row:

Bedrooms	Size	Distance to centre	Neighbourhood	Sold price
3	85 m ²	2 km	Central	?

A classical neural network would start with random weights and train on this dataset. It would repeatedly adjust its internal parameters until it learned, from this table alone, that larger houses tend to be more expensive, central homes carry a premium, and distance from the centre matters.

Tabular foundational models are based on context patterns

A tabular foundation model works differently. Most of the learning has already happened *before* you use it.

During pretraining, the model is exposed to millions of tabular prediction tasks. Not one dataset. Not one schema. Not one business problem. **Many** different synthetic or real tables, with different feature types, target variables, noise patterns, missing values, correlations, and nonlinear relationships. Over time, the model does not just learn one task. It learns something closer to the *shape* of tabular prediction itself.

If we continue with our example above, the foundational model has already seen many house-price-like problems during pretraining. It has seen tables where size matters, where location matters, etc.

So when it sees this new table, it reads the labelled rows as examples of the local rules. It may infer something like:

“In this dataset, size increases price, central neighbourhoods are expensive, and being close to the centre adds value. A 3-bedroom, 85 m² central property 2 km from the centre should probably be priced near the upper-middle part of the table.”

The difference is in-context learning

If we encapsulate what each paradigm is doing in a question like format, these would be:

1. A classical neural network asks: **“What pattern can I learn from this dataset if I train long enough?”**
2. A tabular foundation model asks: **“Given everything I learned from many previous tabular tasks, what pattern does this new dataset**

seem to follow?”

This is why these models are often described as doing *in-context learning*. The training rows are no longer only used to update weights. They become part of the prompt. The model looks at them, infers the relationship between features and target, and produces predictions without necessarily taking a gradient step on your data.

Think of it this way. The traditional approach is like hiring a specialist who trained for years on one domain: strong there, but they need to re-learn from scratch in every new domain. A tabular foundation model is more like a researcher who has worked across dozens of fields — they have not seen your specific problem before, but they have developed a pattern-recognition capacity that transfers. They read your data, identify the structure, and start predicting well without months of domain-specific training.

How did we get from TabPFN to Google TabFM?

Google TabFM converges concept from 4 major advanced in foundational models, borrowing row compression from TabICL and in-context learning from TabPFN, and packaging the result into an officially supported scikit-learn-compatible release. Let's briefly cover these earlier models.

TabPFN — the conceptual breakthrough

The problem TabPFN set out to solve was whether in-context learning was even possible for tabular prediction. Could a model trained on synthetic data

do tabular prediction by treating training rows as context?

The answer, published in [Nature \(Hollmann et al.\)](#), was yes.

TabPFN's architecture processes data in 3 stages.

1. In the first stage, each feature column is embedded independently using a transformer with inducing-point attention — this captures column-level statistics without requiring quadratic attention over all rows.
2. In the second stage, each row is compressed into a small number of CLS-like vectors, reducing the variable-length feature sequence to a fixed-dimension row embedding.
3. In the third stage, an ICL transformer receives all row embeddings — training and test rows together — and test rows attend to training rows to produce predictions.

TabPFN “worked”. On small datasets (up to roughly 10,000 rows), it was competitive with AutoML pipelines in a single forward pass. It was the first clean proof that the in-context learning paradigm transfers to tables. But, the limit was scale. ~10k rows is too small for most production datasets.

TabPFN-2.5 and TabPFN-3 — scaling the line

The obvious next question was whether the approach could be pushed to larger datasets. TabPFN-2.5 and TabPFN-3 scaled the original architecture to handle up to 50k rows and 2k features. This required the Muon optimiser (replacing AdamW) and scalable softmax in attention to manage memory at larger context lengths.

On [TabArena](#) — the most rigorous independent tabular benchmark available — TabPFN-3 in default configuration (single forward pass, no tuning) achieves an Elo of 1673. AutoGluon running a 4-hour ensemble achieves 1695. That is a meaningful gap closed: a single-pass, zero-configuration model matching a heavily-resourced AutoML system to within 1%.

TabICL and TabICLv2 — making in-context-learning scale

TabPFN's approach has an architectural cost. Running full row/column attention over the entire table is $O(T^2)$ in the number of rows. At 10k rows that is manageable; at 100k rows it is not.

[TabICL \(Qu et al., ICML 2025\)](#) addressed this by changing the scaling strategy. Instead of running attention over the raw feature matrix, TabICL first encodes each row into a fixed-dimension embedding, then runs the in-context-learning (a.k.a. ICL) transformer over those row embeddings. This is a much cheaper operation — the cost of building row embeddings is roughly linear in the number of features, and the ICL step then operates on compact vectors rather than full feature sequences.

[TabICLv2 \(Qu et al., Feb 2026\)](#) extended this further: a new synthetic data generation engine, the Muon optimiser, and scalable softmax in attention. TabICLv2 without any tuning surpasses RealTabPFN-2.5 (which uses tuning, ensembling, and real-data fine-tuning) on both TabArena and TALENT. It generalises to million-scale datasets within 50GB of GPU memory. The weights are fully open.

This is the architectural idea Google TabFM directly borrows: row compression to fixed-dimension vectors, followed by an ICL transformer over those vectors.

TabDPT — the real-data branch

A parallel research question ran alongside the scaling work: does pretraining on purely synthetic data generalise well enough?

TabDPT (Xu et al., NeurIPS 2025) explored the alternative. It combines ICL-based retrieval augmentation with self-supervised learning on real tabular corpora, rather than purely synthetic SCM data. The finding was that real tabular data does improve downstream generalisation beyond what synthetic-only priors achieve, and that performance scales predictably with both model size and the quantity of pretraining data.

TabDPT represents the “synthetic priors alone may not be enough” branch of the family tree. Google TabFM does not appear to use real tabular pretraining data — the model card states synthetic SCM pretraining, and the generator code is not public. Whether that is a limitation will become clearer as evaluation data accumulates.

The synthesis

Google TabFM is not a revolution. It is a *synthesis*. It takes the core in-context learning approach from TabPFN, the row compression + ICL scaling architecture from TabICL, and applies synthetic SCM pretraining — the approach proven across the whole family. Understanding that lineage is understanding why TabFM looks the way it does. It is not a fresh start; it is the

field's current best ideas packaged into an official, scikit-learn-compatible release.



How does Google TabFM actually work?

Google TabFM processes a table in 4 sequential stages.

Created with Datawrapper

Stage	Plain-English question
1. Cell embedding	What does each value mean, given nearby columns?
2. Column attention	What does each column look like across the whole dataset?
3. Row attention	What feature combinations matter inside each row?
4. ICL transformer	Given the labelled rows, what should the label be for the test rows?

Created with Datawrapper

Let’s understand this with an example. Suppose we have a customer churn dataset like the one below.

Customer	Monthly spend	Support tickets	Contract type	Tenure	Churned?
A	£20	0	Annual	24 months	No
B	£80	5	Monthly	2 months	Yes
C	£60	4	Monthly	3 months	Yes
D	£45	1	Annual	18 months	No
E	£90	0	Annual	30 months	No
F	£70	4	Monthly	2 months	?

Stage 1: Cell embedding

The first stage turns raw table values into vectors.

But it does not treat each cell completely alone. In TabFM, each feature token can be enriched using a small group of nearby features. So instead of seeing this:

```
Monthly spend = £70
```

The model may embed it with nearby context:

```
Monthly spend = £70  
Support tickets = 4  
Contract type = Monthly
```

That matters because a value rarely means much in isolation. £70 monthly spend could mean different things depending on nearby columns.

Stage 2: Column attention

Now the model looks **down each column**.

This is very different from looking across one customer. It asks: *What kind of column is this?*

- Take the **Support tickets** column. The model can learn that this column has mostly low values, but values around 4 or 5 are unusual.
- Now take **Tenure**. Again, the model can see that 2 months is low compared with most of the table.

This is where column attention is useful. It lets the model understand the distribution of each feature

Stage 3: Row attention

Now the model looks **across features within each row**.

Each feature is informative, but the real signal is in the combination.

- A customer with many support tickets may churn.
- A customer on a monthly contract may churn.
- A customer with short tenure may churn.
- *BUT*, the combination of all three is much stronger

Monthly contract
+ short tenure
+ many support tickets
= high churn risk

This is what row attention captures. It asks: *Which columns matter together for this row?*

Stage 4: Row compression and ICL transformer

By now, each row has a rich representation.

But rows may have many features. The model compresses each row into a fixed number of summary vectors, the CLS tokens.

So instead of carrying every feature token forward, customer F becomes something like:

```
Customer F = [summary vector 1, summary vector 2, ..., summary  
vector 8]
```

This is like compressing a customer profile into a small set of learned summaries.

Now the real in-context learning happens. The model sees labelled rows:

```
Customer A → No churn  
Customer B → Churn  
Customer C → Churn  
Customer D → No churn  
Customer E → No churn
```

And the test row:

Customer F → ?

The ICL transformer lets F attend to A–E.

It may discover:

Customer F looks similar to B and C.
B and C churned.
Therefore F is likely to churn.

This is the first stage where labels matter.

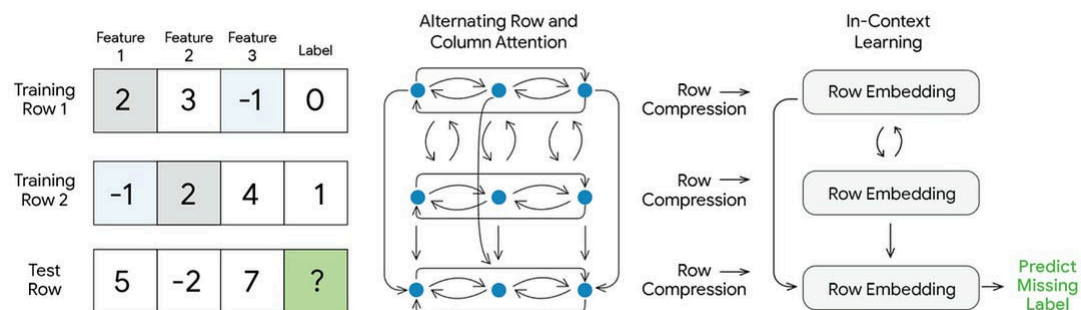
Stages 1–3 understood the table structure, column distributions, and row-level feature interactions. But they did not use Churned?.

Stage 4 uses the labelled examples to infer the prediction rule.

In-context learning happens here in stage 4. Training rows' CLS vectors attend to each other. Test rows' CLS vectors attend to training rows' CLS vectors. The model reads the pattern in the training examples — the relationship between features and labels — and propagates it to the test rows. Predictions emerge from this attention pattern.

The important point: this attention step is the only place where training labels influence the prediction. The model does not receive labels during Stages 1–

3. Only in the ICL transformer do training rows contribute their label information to the test predictions, through the attention mechanism.



Architecture diagram shared by Google in the blog post.

What `.fit()` actually does

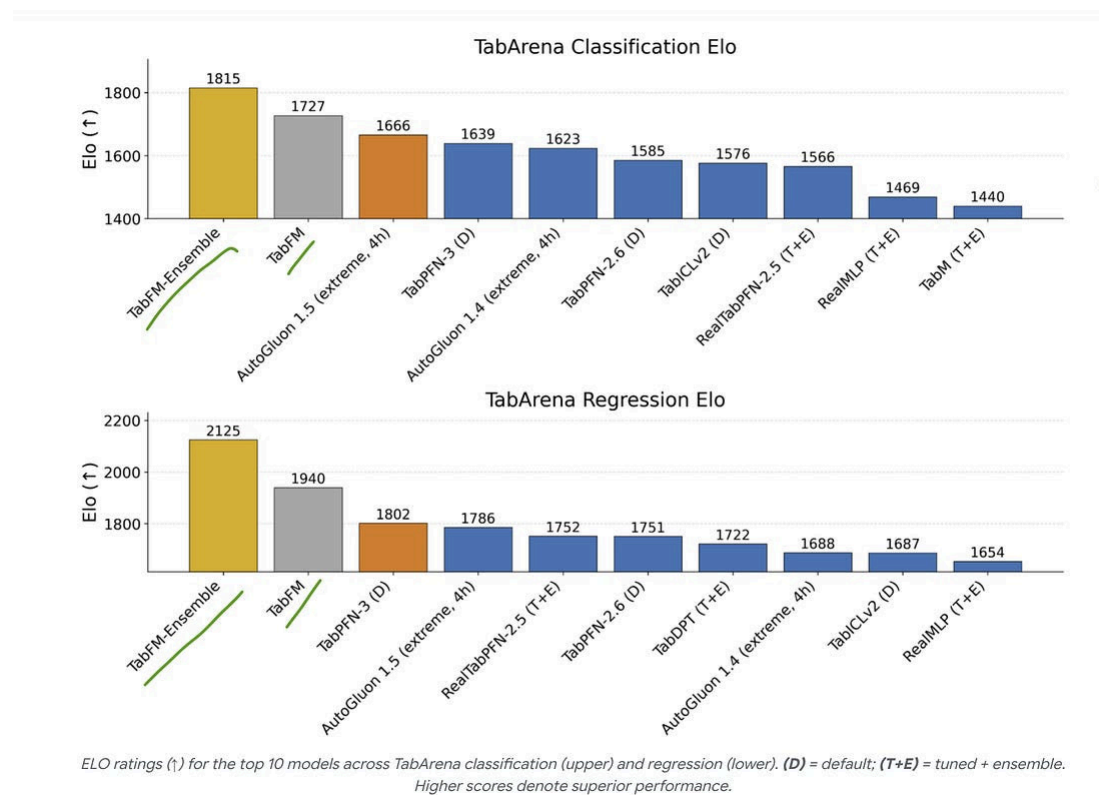
When you call `clf.fit(X_train, y_train)`, no gradient computation happens. The foundation model weights are not updated — not a single parameter changes. What `.fit()` actually does is prepare the column encoders (handling categorical features, missing values) and fit numerical scalers. Then the training rows are stored, to be used as context when `.predict()` is called.

The learning — in the deep learning sense of weight updates — happened entirely during pretraining, on hundreds of millions of synthetic SCM-generated datasets. The generator code is not public. The model card describes the pretraining data as “hundreds of millions of dynamically generated synthetic tabular datasets” and specifies separate classification

and regression checkpoints, trained with cross-entropy and RMSE loss respectively. Exact parameter count is not published.

What is TabFM-Ensemble, and why does it matter for reading the benchmark?

When you read Google's headline benchmark numbers, they are impressive.



They show performance benchmarks using 2 versions of their TabFM architecture. The “simple” one and the ensemble mode one.

We won't cover how the ensemble is built, but for completeness, understand that the base TabFM is the model making 1 prediction, whereas, TabFM-Ensemble is Google asking the same model several slightly different versions of the same question, then combining the answers.

TabFM-Ensemble should be compared with stronger baselines too: tuned XGBoost, tuned CatBoost, LightGBM ensembles, or AutoML-style systems.

The right takeaway is not:

“TabFM magically beats trees in one pass.”

The better takeaway is:

“A pretrained tabular foundation model becomes especially strong when wrapped in familiar ML tricks: ensembling, feature augmentation, learned weighting, and calibration.”

What does the TabArena benchmark evidence actually show for TabFM?

TabArena is the most rigorous public tabular benchmark available right now. Comparing TabFM against it's peer competitor tabular foundational models shows a clear win using TabArenas benchmark.

The **AI Multiple independent benchmark** ran 8 models across 19 datasets with cross-validated folds and produced some of the cleanest third-party numbers available. Across all six size/feature-type regimes it tested, TabFM held the top position, with a mean rank of 1.42 and 15 outright wins. The small-data regime (under 1,000 rows) was the most striking: foundation models occupied the top four slots. In the medium regime (1,000–10,000 rows), TabFM averaged 85.3% ROC-AUC against CatBoost's 83.4%. Even in the large-data regime (above 10,000 rows), TabFM led.

One to highlight though. There is no peer-reviewed TabFM paper at time of writing. The primary evidence for the model's performance comes from Google's own blog post and GitHub repository. That does not make the results wrong — the AI Multiple benchmark is independent confirmation — but it means the claims have not gone through the scrutiny of external reviewers with full access to the training setup and data.

When should you reach for TabFM, and when should you stick with XGBoost?

There is no universal answer, but the evidence points clearly enough that a decision heuristic is defensible.

TabFM is worth trying first when:

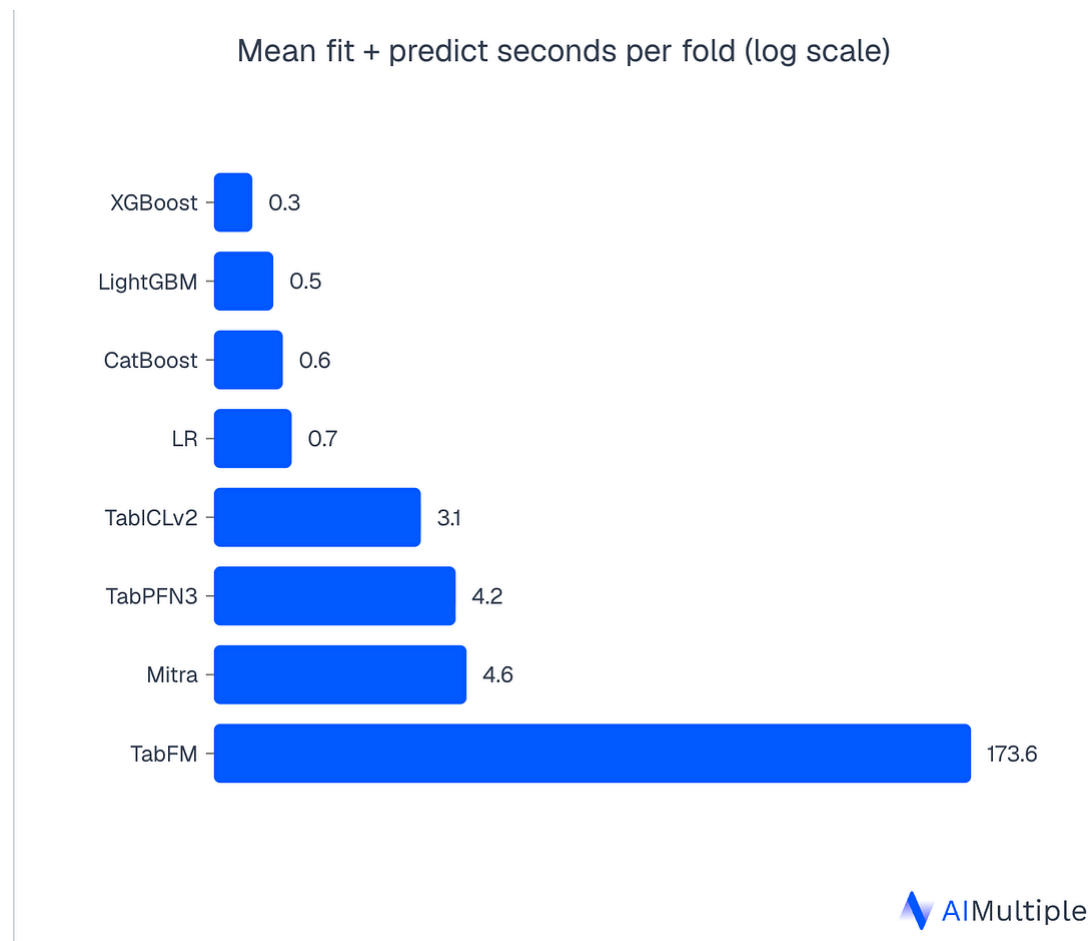
- Your dataset has under roughly 50,000 rows. The advantage is clearest here and shrinks as row count grows.

- Features are mixed numerical and categorical. TabFM handles this natively without extensive preprocessing pipelines.
- You want a strong baseline quickly, without hyperparameter tuning. A single `TabFMClassifier().fit(X, y)` is a legitimate starting point.
- You are working with thin-data segments where GBDTs cannot fit reliably. The [insurance pricing case from Burning Cost](#) shows TabPFN (the direct predecessor) working on fewer than 1,000 policy-years — regimes where GLM confidence intervals are too wide to be actionable. The [KU Leuven xG modelling case](#) found TabPFN reaching parity with a converged logistic regression at around 400 training examples.

Stick with XGBoost, LightGBM, or CatBoost when:

- You need CPU-only deployment or inference under 100ms. TabFM requires a GPU and does not meet real-time latency requirements on any dataset of meaningful size.
- Your dataset has millions of rows. Context length grows with training set size; inference cost scales accordingly.
- You have more than 500 features or more than 10 target classes. Both are hard limits in the released checkpoint.
- Your objective function is custom — non-standard losses, monotonic constraints, business-rule adjustments. GBDTs support these directly; TabFM does not.
- Model governance requires interpretability or regulatory explainability. SHAP values on a GBDT are auditable in a way that attention weights on a frozen transformer are not.

- You cannot absorb a 20-plus-minute JAX compile on cold start. That delay is a real operational cost in a pipeline that restarts frequently.



Therefore, the right frame is not “old vs new.” It is cost/benefit with explicit tradeoffs. TabFM trades compute and deployment complexity for less tuning and stronger small-data performance. That trade is worth making on the right problem. It is the wrong trade on a large, production-scale dataset with strict latency requirements.

Where does tabular ML and foundation model research go from here?

The direction is clear. The pace is not.

Google has announced that TabFM will be available via BigQuery AI . PREDICT SQL integration — foundation models moving into enterprise analytics workflows, usable without Python or a dedicated ML infrastructure team. Whether that integration ships on the announced timeline and what its practical performance looks like at BigQuery scale remain to be seen. But the intent signals where the product roadmap is pointing.

The open-source ecosystem is accelerating independently. [Probabl has documented](#) how TabICLv2 and TabPFN-3 built on each other through shared weights, pretraining code, technical reports, and scikit-learn-compatible APIs — each release extending the reach and capability of the one before. That compounding dynamic is characteristic of healthy research fields, and it is happening faster in tabular ML now than at any previous point.

The practical trajectory for practitioners is probably this: foundation models become the default first-pass baseline in AutoML pipelines. You try them first on any new tabular problem, get a calibrated zero-tuning baseline, and then decide whether the cost/benefit of staying with that baseline — or switching to a tuned GBDT — makes sense for your specific data volume, latency requirements, and compute budget. The decision is not binary and never needs to be.

We are NOT at “replace GBDTs in production” yet. On large datasets with strict latency requirements, the case for GBDTs remains strong. But on small and medium datasets, the foundation model advantage is real, independently confirmed, and growing. The gap is narrowing on a trajectory that has been consistent across every new release in this family since TabPFN v1.



Closing thoughts: the trees aren't dead yet, but they have company

Gradient-boosted trees dominated tabular ML for roughly a decade because they had the right inductive bias for the format. They still do. The case for XGBoost on a large, clean dataset with adequate labels has not materially weakened. What has changed is the regime where that case no longer applies: small datasets, thin segments, rapid prototyping without hyperparameter tuning. In those regimes, the tabular foundation model family has moved from “interesting research result” to “defensible production choice.”

Google TabFM matters not because the synthesis pattern architecture is now available in three lines of Python, with both JAX and PyTorch backends and a HuggingFace model card.

The thesis of this post was stated in the title. A genuine step forward for tabular ML. Not a revolution. Both halves of that claim are now, I hope, defensible.

Now, I want to hear from you

The argument here rests on benchmark evidence and code inspection — but production ML is full of domain-specific surprises that no benchmark captures.

- Have you tried any tabular foundation models on a real dataset? What did you find — do the benchmark advantages hold in practice for your domain, or does something specific to your data break the expected pattern?

- Where have GBDTs held up for you even against tuned foundation models? I am curious whether the latency and interpretability arguments are as decisive as I think they are, or whether the compute cost is actually the blocker in most production conversations.
- What would change your calculus on production use of TabFM or TabPFN — a peer-reviewed paper, a larger context limit, CPU support, or something else entirely?

Share your thoughts, examples, or counterpoints in the comments. I would love to read your opinions 📌

Further reading

If you are interested in more content, here is an article capturing it and organising it all by topics!



All my articles, one convenient place

JOSE PARREÑO GARCIA · OCTOBER 15, 2024

[Read full story](#)

✓ Subscribed

References

- [TabFM: A zero-shot foundation model for tabular data \(Google Research blog, Jun 2026\)](#) — the primary source for TabFM architecture,

benchmark configuration, and BigQuery integration announcement

- [Google TabFM GitHub repository](#) — official code release; source for TabFM-Ensemble wrapper components, hard limits, and production notes
- [Why tree-based models still outperform deep learning on tabular data \(Grinsztajn et al., NeurIPS 2022\)](#) — identifies rotational invariance, uninformative features, and irregular target functions as the structural reasons GBDTs beat deep learning on tables
- [Revisiting deep learning models for tabular data \(Gorishniy et al., NeurIPS 2021\)](#) — introduces FT-Transformer and a strong ResNet baseline; honest conclusion that no single DL architecture was universally superior to trees
- [TabPFN: A transformer that solves small tabular classification problems in a second \(Hollmann et al., Nature 2024\)](#) — establishes in-context learning for tabular prediction; the conceptual breakthrough the whole family builds on
- [TabICL: A tabular foundation model for in-context learning on large data \(Qu et al., ICML 2025\)](#) — introduces row compression to fixed-dimension embeddings followed by an ICL transformer, enabling scaling to 100k training samples
- [TabICLv2: A better, faster, scalable, and open tabular foundation model \(Qu et al., Feb 2026\)](#) — extends TabICL with Muon optimiser and scalable softmax; surpasses RealTabPFN-2.5 without tuning; fully open weights

- [TabDPT: Scaling tabular foundation models \(Xu et al., NeurIPS 2025\)](#) — explores combining ICL with real tabular pretraining data, finding that real data improves generalisation beyond synthetic-only priors
- [TabArena: A living benchmark for machine learning on tabular data \(Erickson et al., arXiv 2506.16791\)](#) — 51 curated IID datasets, 27+ methods; introduces BeyondArena for temporal and grouped splits; key finding that protocol choice affects rankings as much as architecture
- [TabArena GitHub repository \(autogluon/tabarena\)](#) — living benchmark implementation; confirms IID vs BeyondArena split and validation protocol
- [Tabular models benchmark: Performance across 19 datasets 2026 \(AI Multiple\)](#) — independent third-party benchmark; 8 models, 19 datasets; provides concrete latency and GPU cost figures including 173.6s average per fold and \$27 total GPU cost for TabFM
- [Demystifying table foundation models \(Gaël Varoquaux, Probabl, Feb 2026\)](#) — practitioner explainer by the researcher whose lab developed TabICL; covers what TFM's are and what they bring versus GBDTs
- [Open science is powering the tabular foundation models revolution \(Varoquaux & Osborne, Probabl, Jun 2026\)](#) — documents how TabICLv2 and TabPFN-3 built on each other through open weights and scikit-learn APIs
- [Benchmarking TabPFN V2 against XGBoost and CatBoost on Kaggle datasets \(HumbleBeeAI, Mar 2026\)](#) — independent practitioner benchmark on Kaggle competition datasets; confirms small/medium data advantage and scalability limits

- [Prior Labs — Benchmarking TabPFN](#) — step-by-step methodology for head-to-head comparison with XGBoost using ROC-AUC, stratified splits, and matched tuning budgets
- [Tabular foundation models for xG: Can TabPFN score without training? \(DTAI KU Leuven\)](#) — sports analytics case study; TabPFN reaches parity with converged logistic regression at approximately 400 training examples
- [Foundation models for thin segments: TabPFN and TabICLv2 in insurance pricing \(Burning Cost, Mar 2026\)](#) — actuarial use case; demonstrates the small/thin-data advantage in insurance pricing with fewer than 1,000 policy-years

Thanks again for supporting the Senior Data Science Lead newsletter.

Your support makes it possible for me to keep writing openly, while also investing time in the more structured, hands-on work — courses, templates, and tools — that help people apply these ideas in practice. You can find them [here](#).



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

This newsletter exists in its current form because of readers like you. I am genuinely grateful for that.

Thank you.

 LIKE

 COMMENT

 RESTACK

© 2026 Jose Parreño Garcia
[548 Market Street](#) PMB 72296, San Francisco, CA 94104
[Unsubscribe](#)

Get the app

 Start writing